

ОТЧЕТ ПО HW5

Кажкаримов Асхат, БПИ-233, aakazhkarimov@edu.hse.ru

Я претендую на оценку до 10 баллов. Серверная часть реализована на Java 17 с Netty. Клиентская часть реализована на чистом JavaScript.

Генеративный ИИ не был использован для работы над этим заданием.

Дополнительно код работы можно найти по ссылке:

<https://github.com/justcipunz/websocket-chat>

Цель работы

Цель работы - реализовать клиент-серверное приложение чата на основе WebSocket. Клиент должен работать в браузере на чистом JavaScript. Сервер должен быть реализован на Java с использованием Netty.

Взаимодействие клиента и сервера должно происходить через WebSocket Binary Mode. Для бизнес-протокола должны использоваться Protocol Buffers.

В работе реализован чат с несколькими пользователями. Пользователь вводит имя и подключается к чату. После подключения он может отправлять публичные сообщения, приватные сообщения через формат @name text, а также прикреплять изображения.

Изображение, выбранное при подключении, используется как иконка пользователя.

Сервер хранит историю сообщений и изображений в памяти. При подключении нового пользователя сервер отправляет ему видимую историю сообщений и последнее видимое изображение.

Что использовалось

Работа выполнена на Java 17.

Сборка проекта выполняется через Maven.

Для серверной части использовался Netty.

Для бизнес-протокола использовались Protocol Buffers.

Для Java использовалась зависимость protobuf-java.

Для клиентской части использовался чистый JavaScript, без фреймворков.

Для работы с Protocol Buffers в браузере использовался protobuf.js.

Для логирования на сервере использовался slf4j-simple.

Для проверки WebSocket-сообщений использовались DevTools браузера.

Структура программы

Основной код находится в пакете ru.hse.hw5.

Основные пакеты и директории

ru.hse.hw5.netty

Содержит сетевую часть сервера. Здесь создается Netty-сервер, настраивается pipeline и обрабатываются WebSocket frames.

ru.hse.hw5.chat

Содержит бизнес-логику чата. Здесь хранятся активные пользователи, сессии, история сообщений и история изображений.

ru.hse.hw5.protocol

Содержит код для работы с бизнес-протоколом. Здесь выполняется кодирование и декодирование protobuf-сообщений, создание ошибок и валидация запросов.

ru.hse.hw5.model

Содержит enum с кодами ошибок.

src/main/proto

Содержит файл chat.proto. В этом файле описан бизнес-протокол приложения.

src/main/resources/client

Содержит клиентскую часть:

- index.html;
- style.css;
- app.js;
- protobuf/chat_pb.js;
- protobuf/protobuf.min.js.

src/test/java

Содержит тесты для бизнес-логики сервера и проверки валидатора.

Серверная часть

ServerMain

ServerMain является точкой входа в серверное приложение.

В main создается объект ChatService. Он хранит состояние чата и обрабатывает бизнес-запросы.

После этого создается ChatServer. В него передается порт 8080 и ChatServerInitializer.

Сервер запускается командой:

```
mvn exec:java
```

После успешного запуска сервер выводит адрес:

```
ws://localhost:8080/chat
```

Если сервер завершается, в блоке finally вызывается chatServer.stop(). Это нужно, чтобы освободить ресурсы Netty.

ChatServer

ChatServer отвечает за запуск Netty-сервера.

В методе start создаются bossGroup и workerGroup.

bossGroup принимает входящие подключения.

workerGroup обрабатывает сетевые события подключенных клиентов.

Для сервера используется NioServerSocketChannel.

ServerBootstrap настраивается так:

- указывается bossGroup и workerGroup;
- указывается тип канала;
- указывается ChatServerInitializer.

После настройки сервер привязывается к порту 8080.

Метод awaitShutdown ждет закрытия serverChannel.

Метод stop закрывает serverChannel и завершает event loop groups.

ChatServerInitializer

ChatServerInitializer настраивает pipeline для каждого нового подключения.

Pipeline состоит из таких обработчиков:

- HttpServerCodec;
- HttpObjectAggregator;
- WebSocketServerProtocolHandler;
- WebSocketFrameAggregator;
- WebSocketFrameHandler.

HttpServerCodec нужен для обработки HTTP handshake перед переходом на WebSocket.

HttpObjectAggregator собирает HTTP-сообщение в один объект.

WebSocketServerProtocolHandler переводит соединение в режим WebSocket.

Для него задан путь:

/chat

Также для него задан лимит:

$2 * 1024 * 1024$

Это нужно, чтобы WebSocket мог принять protobuf-сообщение с изображением до 1 МБ.

WebSocketFrameAggregator склеивает фрагменты WebSocket-сообщения. Это важно для изображений. Большое binary-сообщение может прийти не одним фреймом, а несколькими частями.

После агрегации WebSocketFrameHandler получает уже целый BinaryWebSocketFrame.

Фрагмент настройки pipeline:

```
WebSocketServerProtocolConfig config =  
WebSocketServerProtocolConfig.newBuilder()
```

```
    .websocketPath("/chat")
```

```
    .maxFramePayloadLength(2 * 1024 * 1024)
```

```
    .build();
```

```
pipeline.addLast(new HttpServerCodec());
```

```
pipeline.addLast(new HttpObjectAggregator(65536));
```

```
pipeline.addLast(new WebSocketServerProtocolHandler(config));
```

```
pipeline.addLast(new WebSocketFrameAggregator(2 * 1024 * 1024));
```

```
pipeline.addLast(new WebSocketFrameHandler(chatService));
```

WebSocketFrameHandler

WebSocketFrameHandler обрабатывает WebSocket frames.

Если приходит BinaryWebSocketFrame, обработчик получает из него байты. Затем эти байты передаются в ProtocolCodec.decodeClientRequest.

Если protobuf-запрос не удалось декодировать, клиенту отправляется ошибка INVALID_REQUEST.

Если запрос декодирован успешно, он передается в ChatService. ChatService возвращает список OutboundMessage.

В каждом OutboundMessage есть канал получателя и ServerResponse.

После этого каждый ServerResponse кодируется в массив байт и отправляется через BinaryWebSocketFrame.

Если клиент отправляет TextWebSocketFrame, сервер возвращает ошибку UNSUPPORTED_FRAME_TYPE. Бизнес-сообщения в текстовом виде не принимаются.

При отключении клиента вызывается chatService.handleDisconnect. Это удаляет сессию пользователя и освобождает имя.

Основная логика обработки binary frame:

```
private void handleBinaryFrame(ChannelHandlerContext ctx,  
BinaryWebSocketFrame frame) {
```

```
    ByteBuf content = frame.content();
```

```
    byte[] bytes = new byte[content.readableBytes()];
```

```
    content.getBytes(content.readerIndex(), bytes);
```

```
    ProtocolCodec.DecodedRequest decoded =  
ProtocolCodec.decodeClientRequest(bytes);
```

```
    if (!decoded.isSuccess()) {
```

```

        writeResponse(ctx, decoded.errorResponse());
    }
    return;
}

List<ChatService.OutboundMessage> out =
    chatService.handleClientRequest(ctx.channel(), decoded.request());

for (ChatService.OutboundMessage message : out) {
    byte[] encoded = ProtocolCodec.encodeServerResponse(message.response());
    message.channel().writeAndFlush(
        new BinaryWebSocketFrame(
            message.channel().alloc().buffer(encoded.length).writeBytes(encoded)
        )
    );
}
}

```

Бизнес-логика чата

ChatService

ChatService содержит основную бизнес-логику приложения.

Внутри ChatService хранятся:

- sessionsByChannelId;
- sessionsByUserName;
- messageIdGenerator;
- messageHistory;
- imageHistory.

sessionsByChannelId связывает Netty ChannelId с сессией пользователя.

sessionsByUserName связывает имя пользователя с сессией.

messageIdGenerator создает уникальные id сообщений. Первое сообщение получает id = 1.

messageHistory хранит последние 50 сообщений.

imageHistory хранит последние 50 сообщений с изображениями.

handleClientRequest

Метод handleClientRequest принимает канал клиента и ClientRequest.

Сначала метод проверяет, что в ClientRequest есть payload. Если payload не задан, сервер возвращает INVALID_REQUEST.

Дальше метод смотрит на тип payload:

- JOIN;
- SEND_MESSAGE.

Если пришел JOIN, вызывается handleJoin.

Если пришел SEND_MESSAGE, вызывается handleSendMessage.

handleJoin

Метод handleJoin обрабатывает подключение пользователя к чату.

Сначала для channel создается или находится ChatSession.

Если сессия уже подключена к чату, сервер возвращает ALREADY_JOINED.

После этого сервер удаляет пробелы в начале и конце имени пользователя. Затем создается нормализованный `JoinRequest`.

Если пользователь передал иконку, она также добавляется в нормализованный запрос.

Дальше запрос проверяется через `ProtocolValidator.validateJoinRequest`.

Проверяются:

- имя не пустое;
- имя не длиннее 20 символов;
- имя соответствует регулярному выражению;
- иконка корректна, если она передана.

Если имя уже занято, сервер возвращает `NAME_ALREADY_TAKEN`.

Если в чате уже 100 пользователей, сервер возвращает `TOO_MANY_USERS`.

Для регистрации пользователя используется `putIfAbsent`. Это нужно, чтобы избежать ошибки при двух одновременных подключениях с одинаковым именем.

После успешной регистрации в `ChatSession` сохраняются:

- имя пользователя;
- флаг `joined`;
- иконка пользователя, если она была передана.

Затем сервер формирует историю сообщений для этого пользователя. Для этого вызывается:

```
messageHistory.visibleFor(normalizedName)
```

Этот метод возвращает только те сообщения, которые пользователь имеет право видеть.

Также сервер ищет последнее видимое изображение:

```
imageHistory.lastVisibleFor(normalizedName)
```

После этого сервер формирует JoinSuccess. В него добавляются:

- assigned_name;
- history;
- last_image, если оно есть.

Ответ отправляется только пользователю, который подключился.

handleSendMessage

Метод handleSendMessage обрабатывает отправку сообщения.

Сначала сервер проверяет, что отправитель уже подключен к чату. Если пользователь еще не выполнил join, сервер возвращает NOT_JOINED.

После этого сервер нормализует текст сообщения через trim.

Если в запросе есть recipient_name, сервер также удаляет пробелы в начале и конце имени получателя.

Если recipient_name есть, но после trim он пустой, сервер возвращает USER_NOT_FOUND.

Затем сервер собирает нормализованный SendMessageRequest и передает его в ProtocolValidator.validateSendMessageRequest.

Проверяются:

- длина текста не больше 25 символов;
- сообщение не пустое, если нет изображения;
- изображение корректно, если оно есть.

Если recipient_name присутствует, сообщение считается приватным.

Для приватного сообщения сервер ищет получателя в sessionsByUsername. Если получателя нет, сервер возвращает USER_NOT_FOUND.

Если сообщение корректное, сервер создает ChatMessage.

В ChatMessage записываются:

- id сообщения;

- имя отправителя;
- текст;
- timestamp_epoch_millis;
- признак private_message;
- recipient_name;
- attachment, если есть изображение;
- sender_icon, если у отправителя есть иконка.

После создания сообщение добавляется в messageHistory.

Если в сообщении есть изображение, оно также добавляется в imageHistory.

Затем сервер формирует ChatMessageEvent.

Для публичного сообщения ChatMessageEvent отправляется всем подключенным пользователям.

Для приватного сообщения ChatMessageEvent отправляется только отправителю и получателю.

Если пользователь отправляет приватное сообщение самому себе, событие отправляется один раз.

handleDisconnect

Метод handleDisconnect вызывается при закрытии WebSocket-соединения.

Сначала сессия удаляется из sessionsByChannelId.

Если пользователь был подключен к чату, его имя удаляется из sessionsByUsername.

После этого имя можно использовать снова.

История сообщений при отключении пользователя не очищается. Сообщения остаются в памяти до вытеснения по лимиту 50 сообщений.

ChatSession

ChatSession хранит данные одного подключенного пользователя.

В сессии есть:

- Netty Channel;
- имя пользователя;
- иконка пользователя;
- признак joined.

Channel нужен для отправки ответов конкретному клиенту.

Имя пользователя нужно для отображения сообщений и поиска получателя приватного сообщения.

Иконка пользователя добавляется к его сообщениям.

Флаг joined показывает, прошел ли пользователь успешное подключение к чату.

ChatMessageHistory

ChatMessageHistory хранит историю сообщений.

Максимальный размер истории - 50 сообщений.

Когда добавляется 51-е сообщение, самое старое сообщение удаляется.

В истории хранятся и публичные, и приватные сообщения.

Метод visibleFor возвращает сообщения, видимые конкретному пользователю.

Правила видимости:

- публичные сообщения видны всем;
- приватное сообщение видно отправителю;
- приватное сообщение видно получателю;
- чужие приватные сообщения не возвращаются.

Методы класса синхронизированы. Это нужно, потому что к истории могут обращаться несколько клиентов.

ImageHistory

ImageHistory хранит историю изображений.

В эту историю попадают только изображения, прикрепленные к сообщениям.

Иконки пользователей в ImageHistory не сохраняются.

Максимальный размер истории изображений - 50.

Метод lastVisibleFor ищет последнее изображение, которое видимо конкретному пользователю.

Поиск идет с конца списка. Если изображение находится в публичном сообщении, оно подходит. Если изображение находится в приватном сообщении, оно подходит только отправителю или получателю.

Так сервер не отправляет пользователю чужое приватное изображение в last_image.

Бизнес-протокол

Бизнес-протокол описан в файле:

src/main/proto/chat.proto

Используется синтаксис proto3.

Вся коммуникация между клиентом и сервером идет через Protocol Buffers.

Клиент отправляет только ClientRequest.

Сервер отправляет только ServerResponse.

В ClientRequest используется oneof payload. Это означает, что в одном запросе может быть только один тип бизнес-запроса.

ClientRequest содержит:

- JoinRequest join;
- SendMessageRequest send_message.

ServerResponse также использует oneof payload.

ServerResponse содержит:

- JoinSuccess join_success;
- ChatMessageEvent message_event;
- ErrorResponse error.

Основные сообщения протокола

ClientRequest

ClientRequest является общим контейнером для запросов клиента.

```
message ClientRequest {  
  
    oneof payload {  
  
        JoinRequest join = 1;  
  
        SendMessageRequest send_message = 2;  
  
    }  
}
```

Если клиент хочет подключиться к чату, он отправляет join.

Если клиент хочет отправить сообщение, он отправляет send_message.

JoinRequest

JoinRequest используется для подключения пользователя.

```
message JoinRequest {  
  
    string name = 1;  
  
    optional ImageData icon = 2;  
  
}
```

Поле name содержит имя пользователя.

Поле icon является optional. Если пользователь выбрал иконку при подключении, она передается в icon. Если иконка не выбрана, поле отсутствует.

SendMessageRequest

SendMessageRequest используется для отправки сообщений.

```
message SendMessageRequest {  
    string text = 1;  
    optional ImageData image = 2;  
    optional string recipient_name = 3;  
}
```

Поле text содержит текст сообщения.

Поле image содержит изображение, если оно было прикреплено.

Поле recipient_name содержит имя получателя приватного сообщения.

Если recipient_name отсутствует, сообщение считается публичным.

Если recipient_name присутствует, сообщение считается приватным.

Сервер не парсит символ @ в тексте сообщения. Формат @name text разбирает клиент. В protobuf-запрос сервер получает уже отдельное поле recipient_name.

ServerResponse

ServerResponse является общим контейнером для ответов сервера.

```
message ServerResponse {  
  oneof payload {  
    JoinSuccess join_success = 1;  
    ChatMessageEvent message_event = 2;  
    ErrorResponse error = 3;  
  }  
}
```

JoinSuccess отправляется после успешного подключения.

ChatMessageEvent отправляется при новом сообщении.

ErrorResponse отправляется при ошибке.

JoinSuccess

JoinSuccess отправляется клиенту после успешного подключения к чату.

```
message JoinSuccess {  
  string assigned_name = 1;  
  History history = 2;  
  optional ImageData last_image = 3;  
}
```

assigned_name содержит имя пользователя после обработки сервером.

history содержит видимую историю сообщений.

last_image содержит последнее видимое изображение, если оно есть.

History

History хранит список сообщений.

```
message History {  
    repeated ChatMessage messages = 1;  
}
```

Поле messages является repeated. Оно содержит сообщения в порядке от старых к новым.

ChatMessageEvent

ChatMessageEvent используется для отправки нового сообщения клиентам.

```
message ChatMessageEvent {  
    ChatMessage message = 1;  
}
```

Внутри находится ChatMessage, сформированный сервером.

ChatMessage

ChatMessage является серверной моделью сообщения.

```
message ChatMessage {  
    int64 id = 1;  
    string sender_name = 2;  
    string text = 3;
```

```
int64 timestamp_epoch_millis = 4;  
bool private_message = 5;  
string recipient_name = 6;  
optional Attachment attachment = 7;  
optional ImageData sender_icon = 8;
```

```
message Attachment {  
    ImageData image = 1;  
}  
}
```

id задается сервером.

sender_name содержит имя отправителя.

text содержит текст сообщения после trim.

timestamp_epoch_millis содержит время создания сообщения.

private_message показывает, является ли сообщение приватным.

recipient_name содержит имя получателя приватного сообщения.

attachment содержит изображение сообщения.

sender_icon содержит иконку отправителя.

Внутри ChatMessage есть вложенное сообщение Attachment. Оно используется для прикрепленного изображения.

ImageData

ImageData используется для передачи изображений.

```
message ImageData {  
    string file_name = 1;  
    string mime_type = 2;  
    bytes data = 3;  
    int32 size_bytes = 4;  
}
```

file_name содержит имя файла.

mime_type содержит MIME-тип.

data содержит бинарные данные изображения.

size_bytes содержит размер изображения в байтах.

ErrorResponse

ErrorResponse используется для ошибок.

```
message ErrorResponse {  
    int32 code = 1;  
    string error = 2;  
    string message = 3;  
}
```

code содержит числовой код ошибки.

error содержит машинное имя ошибки.

message содержит текст ошибки для пользователя.

Ошибки и валидация

ErrorCode

ErrorCode содержит все ошибки бизнес-протокола.

Используются такие ошибки:

- UNKNOWN_ERROR - непредвиденная ошибка;
- INVALID_REQUEST - некорректный protobuf-запрос;
- UNSUPPORTED_FRAME_TYPE - клиент отправил не binary frame;
- NOT_JOINED - клиент отправил сообщение до join;
- ALREADY_JOINED - клиент повторно отправил join;
- INVALID_NAME - некорректное имя;
- NAME_ALREADY_TAKEN - имя уже занято;
- EMPTY_MESSAGE - сообщение пустое;
- MESSAGE_TOO_LONG - текст длиннее 25 символов;
- INVALID_IMAGE - некорректное изображение;
- USER_NOT_FOUND - получатель приватного сообщения не найден;
- TOO_MANY_USERS - в чате уже 100 пользователей.

ErrorFactory

ErrorFactory создает ответы с ошибкой.

Метод error принимает ErrorCode и текст ошибки. После этого создается ErrorResponse.

В ErrorResponse записываются:

- числовой код;
- машинный код;
- текст ошибки.

После этого ErrorResponse оборачивается в ServerResponse.

Так все ошибки отправляются клиенту в одном формате.

ProtocolValidator

ProtocolValidator проверяет бизнес-ограничения.

Для JoinRequest проверяется имя пользователя.

Имя считается корректным, если:

- оно не пустое;
- длина не больше 20 символов;
- оно соответствует регулярному выражению.

Регулярное выражение разрешает латинские буквы, русские буквы, цифры, нижнее подчеркивание и дефис.

Если в JoinRequest есть icon, он тоже проверяется.

Для SendMessageRequest проверяется текст сообщения и изображение.

Текст сообщения не должен быть длиннее 25 символов.

Пустой текст разрешен только если к сообщению прикреплено изображение.

Изображение считается корректным, если:

- MIME-тип равен image/jpeg или image/png;
- size_bytes больше 0;
- size_bytes не больше 1 048 576;
- data не пустой;
- размер data совпадает с size_bytes.

Сервер не декодирует изображение как картинку. Для задания достаточно проверить тип, размер и наличие бинарных данных.

ProtocolCodec

ProtocolCodec отвечает за кодирование и декодирование protobuf-сообщений.

Метод decodeClientRequest принимает массив байт и пытается распарсить его как ClientRequest.

Если парсинг успешен, возвращается DecodedRequest.success.

Если парсинг неуспешен, возвращается `DecodedRequest.failure` с ошибкой `INVALID_REQUEST`.

Метод `encodeServerResponse` переводит `ServerResponse` в массив байт.

Этот массив байт затем отправляется клиенту внутри `BinaryWebSocketFrame`.

Клиентская часть

Запуск из папки проекта:

```
npx serve src/main/resources/client
```

index.html

`index.html` содержит одну HTML-страницу клиента.

На странице есть:

- поле имени пользователя;
- кнопка выбора иконки;
- кнопка присоединения;
- статус подключения;
- поле ошибок;
- область сообщений;
- поле ввода сообщения;
- кнопка прикрепления изображения;
- кнопка отправки;
- блок последнего видимого изображения.

Клиент написан на чистом JS и не использует фреймворков.

style.css

`style.css` содержит стили страницы.

В нем описаны:

- основная область страницы;
- панели;
- строки с кнопками;

- область сообщений;
- карточки сообщений;
- ошибки;
- скрытый блок hidden.

app.js

app.js содержит основную логику клиента.

В файле хранятся:

- выбранная иконка пользователя;
- выбранное изображение сообщения;
- WebSocket;
- имя пользователя для join;
- признак успешного join;
- таймеры reconnect;
- список object URL для изображений.

Клиент управляет состояниями интерфейса:

- начальное состояние;
- подключение;
- успешное подключение;
- ожидание reconnect.

До успешного подключения поле сообщения, кнопка отправки и кнопка прикрепления изображения заблокированы.

После JoinSuccess эти элементы становятся активными.

Функция validateImageFile проверяет изображение на стороне клиента.

Проверяются:

- MIME-тип;
- размер не больше 1 МБ.

Функция buildImageDataMessage читает файл через File.arrayBuffer. После этого создается ImageData.

Функция `connectAndJoin` открывает `WebSocket` по адресу:

`ws://localhost:8080/chat`

После открытия соединения клиент задает:

`socket.binaryType = "arraybuffer"`

После события `open` клиент отправляет `JoinRequest`.

Функция `sendJoinRequest` собирает `ClientRequest` с полем `join`, кодирует его через `ChatProto.encodeClientRequest` и отправляет через `socket.send`.

Функция `handleServerResponseArrayBuffer` обрабатывает ответы сервера.

Она декодирует `ArrayBuffer` как `ServerResponse`.

Если пришел `join_success`, клиент отображает историю и последнее видимое изображение.

Если пришел `message_event`, клиент добавляет новое сообщение в область сообщений.

Если пришел `error`, клиент показывает ошибку.

Функция `parsePrivateInput` разбирает приватное сообщение.

Например, строка:

`@Bob hello`

превращается в:

- `recipient_name = Bob;`
- `text = hello.`

Символ `@` на сервер не отправляется.

Функция `sendMessage` отправляет публичные и приватные сообщения.

Если сообщение публичное, в `protobuf` нет `recipient_name`.

Если сообщение приватное, recipient_name заполняется.

Если прикреплено изображение, оно добавляется в поле image.

После отправки поле ввода и выбранное изображение очищаются.

Также в клиенте реализован reconnect. Если соединение не удалось установить или оно закрылось, клиент ждет 10 секунд и пробует подключиться снова.

protobuf/chat_pb.js

Файл chat_pb.js содержит описание protobuf-сообщений для браузера.

В нем используется protobuf.js.

Схема парсится с параметром keepCase: true. Это нужно, чтобы имена полей совпадали с chat.proto.

Например:

- send_message;
- recipient_name;
- file_name;
- mime_type.

Файл создает глобальный объект window.ChatProto.

В нем есть методы:

- encodeClientRequest;
- decodeServerResponse.

encodeClientRequest проверяет и кодирует ClientRequest.

decodeServerResponse декодирует бинарный ответ сервера.

protobuf/protobuf.min.js

Файл protobuf.min.js содержит runtime protobuf.js.

Он нужен клиенту для кодирования и декодирования Protocol Buffers в браузере.

Обмен сообщениями

Подключение пользователя

Сначала пользователь вводит имя и нажимает кнопку “Присоединиться”.

Клиент открывает WebSocket-соединение.

После события open клиент отправляет ClientRequest с payload join.

Сервер получает бинарный WebSocket frame, декодирует protobuf и передает JoinRequest в ChatService.

Если имя корректное и свободное, сервер регистрирует пользователя.

После этого сервер отправляет JoinSuccess. В ответе есть имя пользователя, история сообщений и последнее видимое изображение.

Если имя занято, сервер отправляет ErrorResponse с ошибкой NAME_ALREADY_TAKEN.

Публичное сообщение

Публичное сообщение отправляется через SendMessageRequest без recipient_name.

Клиент кодирует запрос в protobuf и отправляет binary frame.

Сервер создает ChatMessage и сохраняет его в истории.

После этого сервер отправляет ChatMessageEvent всем подключенным пользователям, включая отправителя.

Приватное сообщение

Приватное сообщение вводится на клиенте в формате:

@Bob hello

Клиент разбирает эту строку сам.

На сервер отправляется не строка @Bob hello, а структурированный protobuf-запрос:

- text = hello;
- recipient_name = Bob.

Сервер не ищет символ @ в тексте.

Если пользователь Bob есть в чате, сервер отправляет сообщение только отправителю и Bob.

Если пользователя Bob нет, сервер отправляет ошибку USER_NOT_FOUND.

Изображение в сообщении

Клиент может прикрепить JPEG или PNG до 1 МБ.

Перед отправкой клиент проверяет тип и размер файла.

Затем файл читается через File.arrayBuffer.

Байты файла записываются в ImageData.data.

На сервере изображение снова проверяется. Проверяется MIME-тип, размер и совпадение size_bytes с длиной data.

Если изображение корректно, оно добавляется в ChatMessage.attachment.

Если сообщение публичное, изображение видят все пользователи.

Если сообщение приватное, изображение видят только отправитель и получатель.

Иконка пользователя

При подключении пользователь может выбрать изображение как иконку.

Иконка передается в JoinRequest.icon.

Сервер сохраняет иконку в ChatSession.

Когда этот пользователь отправляет сообщение, сервер добавляет иконку в `ChatMessage.sender_icon`.

Клиенты отображают иконку рядом с именем отправителя.

Иконка не сохраняется в `ImageHistory` и не считается сообщением с изображением.

История сообщений и изображений

Сервер хранит последние 50 сообщений.

Также сервер хранит последние 50 изображений, которые были прикреплены к сообщениям.

При подключении нового пользователя сервер отправляет ему историю сообщений. Перед отправкой история фильтруется по правилам видимости.

Публичные сообщения видны всем.

Приватные сообщения видны только отправителю и получателю.

По такому же правилу сервер выбирает `last_image`. Если последнее изображение было отправлено в чужом приватном сообщении, новый пользователь его не получит.

Ключевые фрагменты реализации

Фрагмент бизнес-протокола

```
message ClientRequest {  
    oneof payload {  
        JoinRequest join = 1;  
        SendMessageRequest send_message = 2;  
    }  
}
```

```
message ServerResponse {  
  oneof payload {  
    JoinSuccess join_success = 1;  
    ChatMessageEvent message_event = 2;  
    ErrorResponse error = 3;  
  }  
}
```

Этот фрагмент показывает, что у клиента есть два вида запросов, а у сервера три вида ответов.

Фрагмент сообщения чата

```
message ChatMessage {  
  int64 id = 1;  
  string sender_name = 2;  
  string text = 3;  
  int64 timestamp_epoch_millis = 4;  
  bool private_message = 5;  
  string recipient_name = 6;  
  optional Attachment attachment = 7;  
  optional ImageData sender_icon = 8;
```

```
message Attachment {  
    ImageData image = 1;  
}  
}
```

В ChatMessage хранится текст сообщения, отправитель, время, признак приватности, получатель, изображение и иконка отправителя.

Фрагмент описания изображения

```
message ImageData {  
    string file_name = 1;  
    string mime_type = 2;  
    bytes data = 3;  
    int32 size_bytes = 4;  
}
```

Изображение передается как bytes. Также передается имя файла, MIME-тип и размер.

Фрагмент Netty pipeline

```
pipeline.addLast(new HttpServerCodec());  
pipeline.addLast(new HttpObjectAggregator(65536));  
pipeline.addLast(new WebSocketServerProtocolHandler(config));  
pipeline.addLast(new WebSocketFrameAggregator(2 * 1024 * 1024));
```

```
pipeline.addLast(new WebSocketFrameHandler(chatService));
```

Этот pipeline сначала обрабатывает HTTP handshake, затем переводит соединение в WebSocket, склеивает фрагменты WebSocket-сообщений и передает готовые frames в обработчик приложения.

Фрагмент отправки JoinRequest на клиенте

```
async function sendJoinRequest() {  
  
    const joinPayload = await buildJoinRequestPayload(pendingJoinName,  
selectedIconFile);  
  
    const bytes = window.ChatProto.encodeClientRequest(joinPayload);  
  
    socket.send(bytes);  
  
}
```

Клиент не отправляет JSON. Он кодирует ClientRequest в массив байт и отправляет его через WebSocket.

Фрагмент настройки WebSocket на клиенте

```
socket = new WebSocket("ws://localhost:8080/chat");  
  
socket.binaryType = "arraybuffer";  
  
socket.addEventListener("open", async () => {  
  
    await sendJoinRequest();  
  
});
```

Клиент явно включает binary mode через `binaryType = "arraybuffer"`.

Фрагмент отправки сообщения

```
const sendMessagePayload = {text: parsed.text};

if (parsed.isPrivate) {
    sendMessagePayload.recipient_name = parsed.recipientName;
}

if (selectedMessageImageFile) {
    sendMessagePayload.image = await
    buildImageDataMessage(selectedMessageImageFile);
}

const bytes = window.ChatProto.encodeClientRequest({
    send_message: sendMessagePayload
});

socket.send(bytes);
```

Если сообщение приватное, клиент заполняет `recipient_name`. Если прикреплено изображение, клиент заполняет `image`.

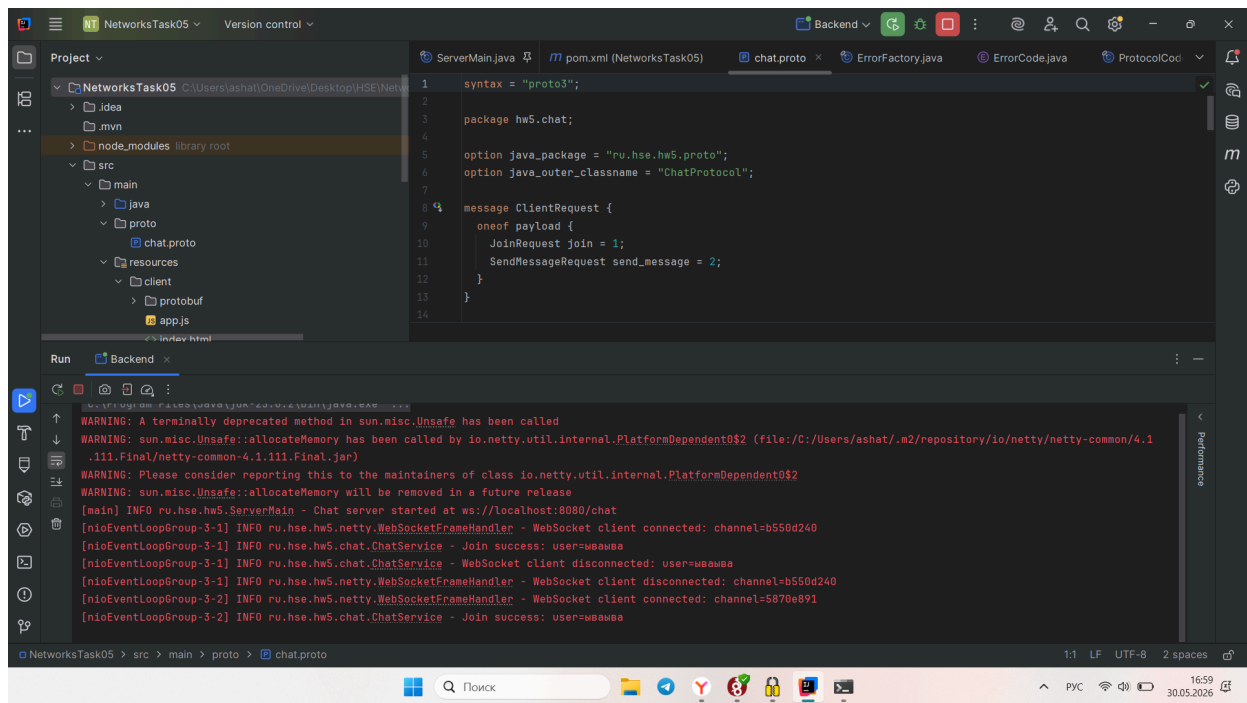
Подтверждение работоспособности

Подробная демонстрация работы приложения записана в видео.

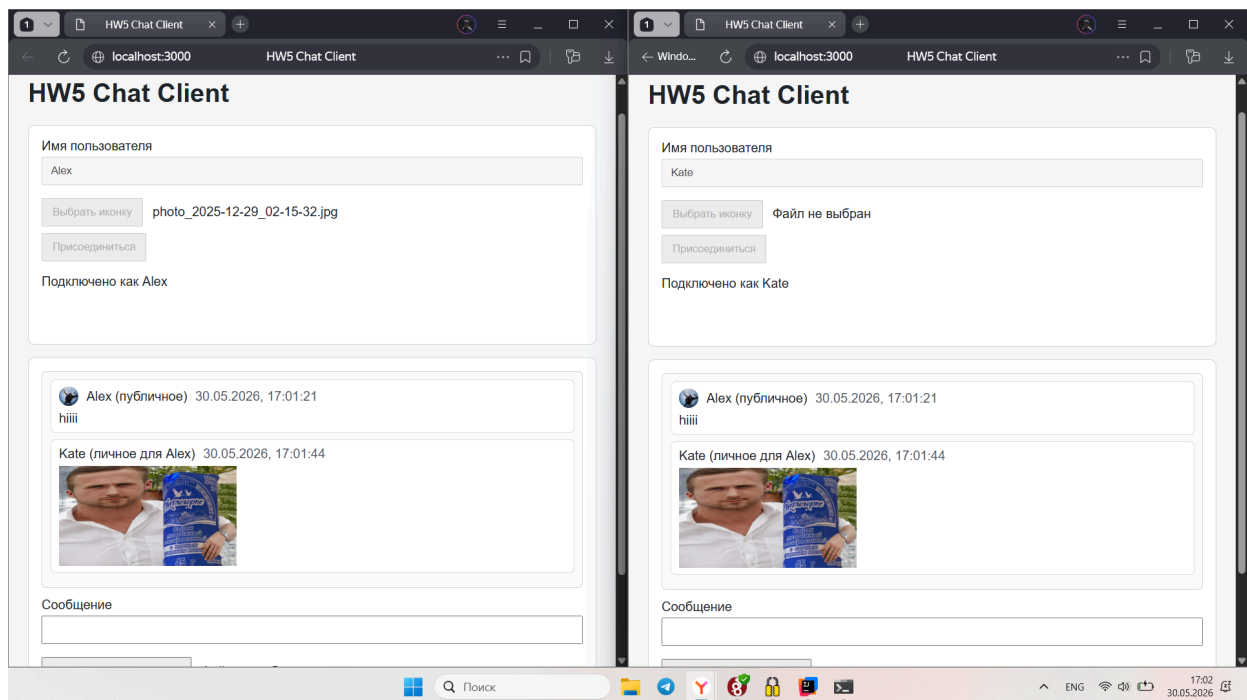
В видео показаны:

- запуск сервера;
- запуск клиентской страницы;
- подключение пользователя по имени;
- ошибка при повторном имени;
- подключение нескольких пользователей;
- публичные сообщения;
- приватные сообщения через @name;
- ошибка при отправке приватного сообщения неизвестному пользователю;
- отправка изображения;
- отклонение изображения неверного типа или размера;
- иконка пользователя;
- история сообщений;
- последнее видимое изображение;
- повторное подключение через 10 секунд;
- проверка WebSocket Binary Mode в DevTools.

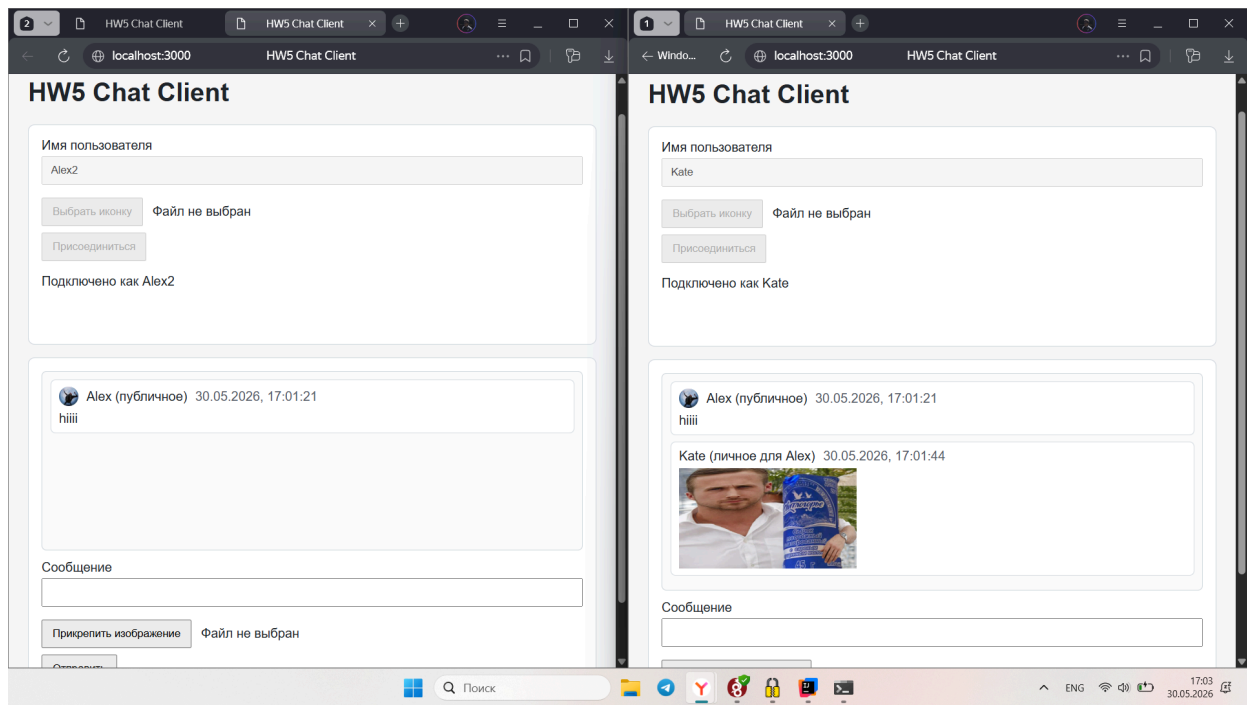
На следующем скриншоте показан запуск сервера и сообщение Chat server started at ws://localhost:8080/chat.



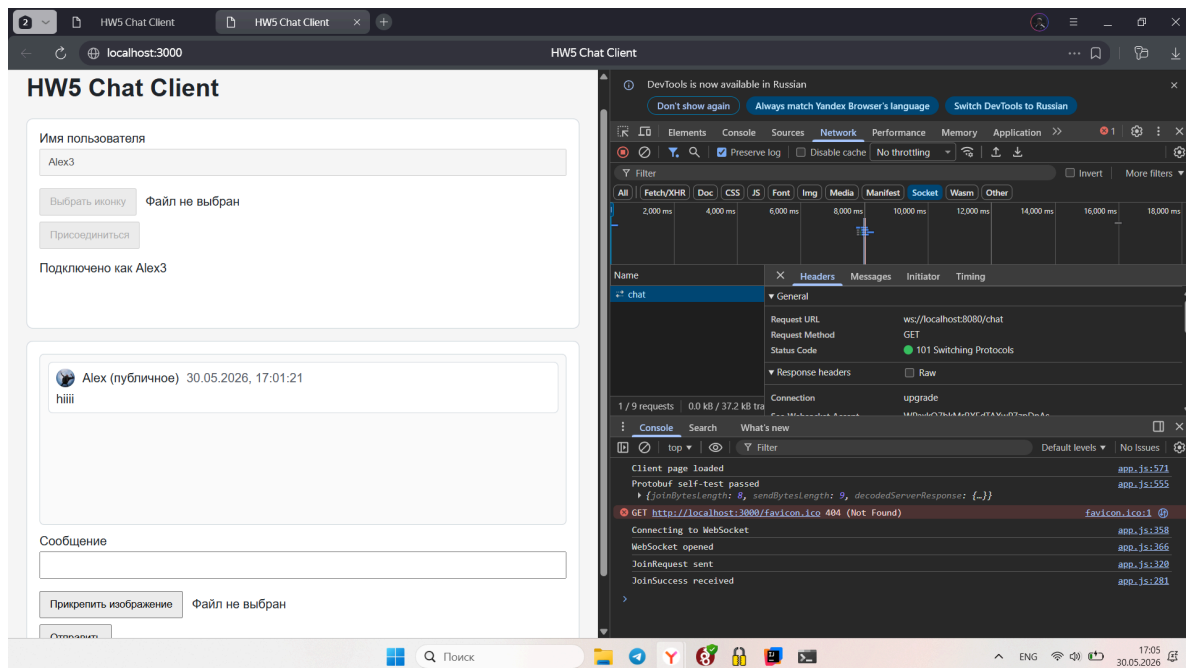
На следующем скриншоте показаны несколько открытых клиентов в браузере.

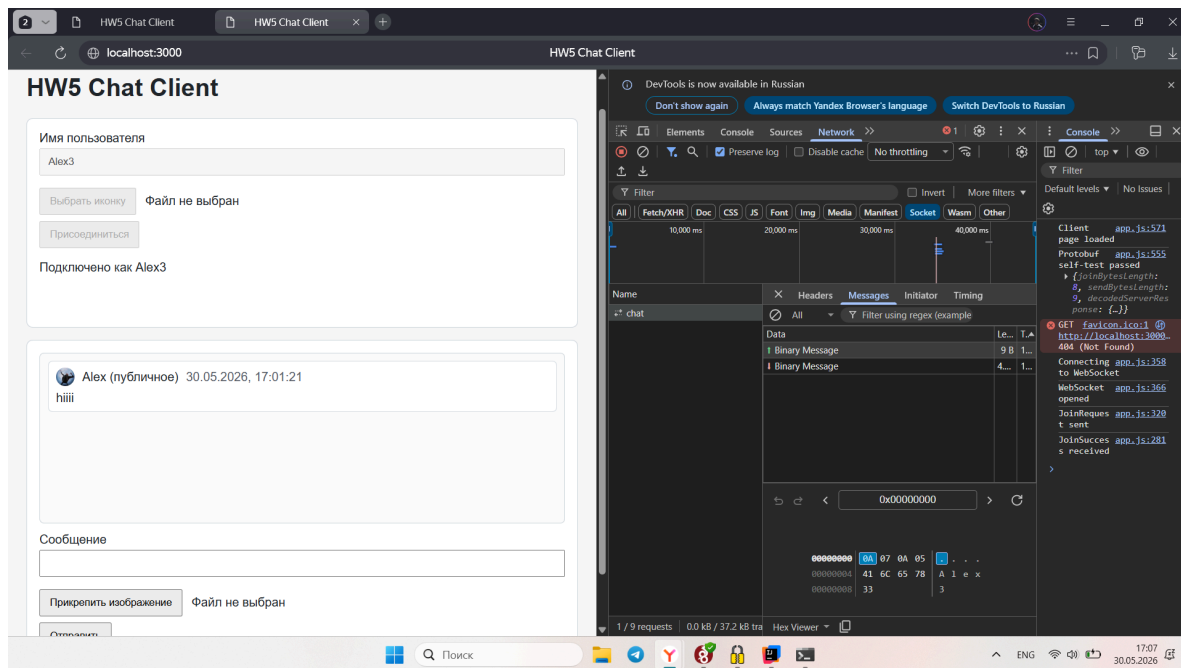


На следующем скриншоте показано, что приватное сообщение видно только отправителю и получателю.



На следующем скриншоте показано WebSocket-соединение в DevTools и binary frames.





Итог

В работе реализовано клиент-серверное приложение чата.

Сервер написан на Java с использованием Netty. Клиент написан на чистом JavaScript.

Клиент и сервер взаимодействуют через WebSocket Binary Mode.

Бизнес-протокол реализован на Protocol Buffers. Для всех бизнес-сообщений используются ClientRequest и ServerResponse.

Реализовано:

- подключение пользователя по имени;
- проверка уникальности имени;
- поддержка до 100 пользователей;
- публичные сообщения;
- приватные сообщения через @name;
- ошибка при отсутствии получателя;
- прикрепление изображений JPEG/PNG до 1 МБ;
- иконка пользователя при подключении;
- история последних 50 сообщений;

- история последних 50 изображений;
- последнее видимое изображение;
- фильтрация приватных сообщений в истории;
- обработка ошибок через ErrorResponse;
- повторное подключение клиента через 10 секунд;
- передача сообщений в binary mode без JSON.

Состояние чата хранится в памяти сервера. База данных и файловое хранилище не используются. После перезапуска сервера история и активные пользователи очищаются.